Stato	Completato
Iniziato	giovedì, 2 luglio 2026, 09:47
Terminato	giovedì, 2 luglio 2026, 09:58
Tempo impiegato	10 min. 42 secondi

Domanda 1

Risposta errata

Esegui `docker run -v ./data:/app/data my-image`. Quali affermazioni sono vere?

a. La cartella `./data` sull'host sovrascrive (oscura) il contenuto di `/app/data` nel container al momento dell'avvio ✓

b. Le cartelle `./data` e `/app/data` diventano read-only

c. La cartella `/app/data` nel container sovrascrive (oscura) il contenuto di `./data` sull'host al momento dell'avvio

La risposta corretta è: La cartella `./data` sull'host sovrascrive (oscura) il contenuto di `/app/data` nel container al momento dell'avvio

Domanda 2

Risposta corretta

Quali affermazioni su CLR e CTS sono corrette?

a. CLR è l'ambiente di esecuzione (runtime) per codice .NET ✓

b. CLR compila tutto sempre ahead-of-time e mai JIT

c. CTS definisce regole/definizioni dei tipi che abilitano interoperabilità tra linguaggi .NET ✓

d. CTS è un garbage collector

Le risposte corrette sono: CLR è l'ambiente di esecuzione (runtime) per codice .NET, CTS definisce regole/definizioni dei tipi che abilitano interoperabilità tra linguaggi .NET

Domanda 3

Risposta corretta

In un workflow GitHub Actions, hai tre job: `build`, `test` e `deploy`, definiti dalla configurazione seguente:

```
jobs:
  build:
    ...
  test:
    needs: build
    ...
  deploy:
    needs: build
    ...
```

Quali job verranno eseguiti e in che ordine?

a. Sequenziale: `build`, poi `test`, poi `deploy`

b. `build` parte per primo. Se finisce con successo, partirà `test`. Se finisce con successo, verrà eseguito `deploy`

c. `build` parte per primo. Se finisce con successo, `test` e `deploy` vanno in parallelo ✓

d. Se `build` fallisce, `test` e `deploy` non verranno eseguiti ✓

e. `build`, `test` e `deploy` partono tutti contemporaneamente

Le risposte corrette sono: `build` parte per primo. Se finisce con successo, `test` e `deploy` vanno in parallelo, Se `build` fallisce, `test` e `deploy` non verranno eseguiti

Domanda 4

Risposta corretta

Che cos'è la **Reflection** in .NET e quali sono i suoi casi d'uso tipici?

a. Permette di caricare assembly di cui non si conosceva l'esistenza al momento della compilazione, abilitando architetture a *plugin* ✓

b. È una tecnica per ottimizzare le prestazioni: il codice che usa la Reflection è molto più veloce dell'accesso diretto ai metodi

c. È un meccanismo che permette a un programma di ispezionare i propri metadati (tipi, metodi, attributi) a runtime ✓

d. Viene utilizzata dai framework di Test, come xUnit, per trovare ed eseguire i metodi marcati con attributi specifici ✓

Le risposte corrette sono: È un meccanismo che permette a un programma di ispezionare i propri metadati (tipi, metodi, attributi) a runtime, Permette di caricare assembly di cui non si conosceva l'esistenza al momento della compilazione, abilitando architetture a *plugin*, Viene utilizzata dai framework di Test, come xUnit, per trovare ed eseguire i metodi marcati con attributi specifici

Domanda 5

Parzialmente corretta

Stai scrivendo un Dockerfile. Devi inserire i file del codice sorgente nell'immagine. Perché la documentazione e le best practice suggeriscono di usare `COPY . .` invece di `ADD . .` se non hai esigenze particolari?

a. Perché `ADD` ha funzionalità aggiuntive (come scaricare file da URL remoti o decomprimere archivi locali) che rendono il comportamento meno prevedibile se non strettamente necessarie ✓

b. Perché `ADD` funziona solo in modalità rootless

c. Non c'è differenza, sono alias dello stesso comando

d. Perché `COPY` crea un layer più piccolo rispetto ad `ADD`

La risposta corretta è: Perché `ADD` ha funzionalità aggiuntive (come scaricare file da URL remoti o decomprimere archivi locali) che rendono il comportamento meno prevedibile se non strettamente necessarie

Domanda 6

Risposta corretta

In uno scenario open-source con fork, quali ruoli hanno tipicamente `origin` e `upstream`?

a. `upstream` è il repository originale ✓

b. `origin` è il repository originale

c. `upstream` è il tuo fork

d. `origin` è il tuo fork ✓

Le risposte corrette sono: `origin` è il tuo fork, `upstream` è il repository originale

Domanda 7

Parzialmente corretta

Sei sul branch `feature` e vuoi integrare le ultime modifiche fatte sul branch `main` (che è andato avanti nel frattempo). Decidi di usare `git rebase main`. Quali delle seguenti affermazioni descrivono correttamente cosa accade al tuo branch `feature`?

a. Git "stacca" i commit del tuo branch `feature` e li riapplica uno alla volta sopra gli ultimi commit di `main` ✓

b. La storia risultante sarà lineare, come se avessi sviluppato la `feature` partendo dall'ultimo commit di `main` ✓

c. Questa operazione è sicura da eseguire anche se il branch `feature` è un branch pubblico condiviso su cui stanno lavorando altri colleghi

d. Git crea un nuovo "Merge Commit" che unisce la storia di `main` e `feature`.

Le risposte corrette sono: Git "stacca" i commit del tuo branch `feature` e li riapplica uno alla volta sopra gli ultimi commit di `main`, La storia risultante sarà lineare, come se avessi sviluppato la `feature` partendo dall'ultimo commit di `main`

Domanda 8

Risposta corretta

In C#, i tipi “primitivi” (es. `int`) sono…

a. alias di tipi del namespace `System` (es. `System.Int32`) ✓

b. indipendenti dal Common Type System

c. tipi speciali che non hanno alcun corrispondente nel CLR

d. sempre tipi riferimento

La risposta corretta è: alias di tipi del namespace `System` (es. `System.Int32`)

Fine revisione